Evaluación de desempeño del ESP32-C6 como controlador para el PiL de MARLUP

Aarón Salas Alvarado, Mónica Solís Murillo, Eduardo Valle Arriola
Instituto Tecnológico de Costa Rica

***Resumen*—Se evaluó un ESP32-C6 (RISC-V de un núcleo a 160 MHz, sin unidad de punto flotante) como posible controlador del lazo de posición de MARLUP. Un firmware de prueba midió, con el contador de ciclos del CPU, la aritmética básica, la duración de un paso del controlador PI del proyecto, el lazo periódico a frecuencias de 1 kHz a 1 MHz, la velocidad del ADC y el tiempo de ida y vuelta de una trama de seis bytes por USB. Un paso del PI tarda 2,66 µs en `float` y 4,51 µs en `double`; a los 100 Hz de MARLUP ocupa el 0,027 % del periodo. El lazo con el PI en `float` se sostiene sin plazos perdidos hasta 200 kHz, pero al incluir la lectura del ADC el tope baja a unos 25 kHz, porque cada lectura tarda 36,7 µs. El tiempo de ida y vuelta por USB es de 0,156 ms en promedio, muy por debajo del periodo de 10 ms. El microcontrolador tiene capacidad de sobra para el lazo de MARLUP; su limitación práctica es que el flujo PIL de Simulink no lo soporta.**

***Index Terms*—ESP32-C6, benchmark, microcontrolador, tiempo real, Processor-in-the-Loop.**

I. Introducción

El PiL de MARLUP se validó con un Arduino UNO R3 (ATmega328P, 8 bits, 16 MHz). Para trabajos posteriores, como un PiL en tiempo real o el control de las tres patas, interesa saber si un microcontrolador más capaz y compacto, el ESP32-C6, tiene margen suficiente. Este reporte mide su desempeño en las operaciones que importan para el lazo: aritmética de punto flotante, el propio controlador PI, la ejecución periódica, la lectura del ADC y la comunicación con la PC.

II. Hardware y método

II-A. Microcontrolador

Los datos del chip se leyeron con `esptool` 5.6.0 (Tabla I). Se trata de un ESP32-C6FH4 con la memoria flash integrada en el encapsulado, lo que permite placas muy pequeñas. El núcleo RISC-V no tiene unidad de punto flotante, por lo que las operaciones con `float` y `double` se resuelven por software.

Tabla I
Datos del chip leídos con `esptool`

Dato	Valor
Chip	ESP32-C6FH4 (QFN32), revisión v0.1
CPU	RISC-V, un núcleo a 160 MHz + núcleo de bajo consumo
Memoria flash	4 MB, integrada en el chip
Cristal	40 MHz
Radio	Wi-Fi 6, Bluetooth 5 (LE), IEEE 802.15.4
USB	USB-Serial/JTAG nativo

II-B. Firmware y medición

El firmware se compiló con el core de Arduino para ESP32 3.3.12 (placa *ESP32C6 Dev Module*, 160 MHz, *USB CDC On Boot* activado). Todos los tiempos se miden con el contador de ciclos del CPU, cuya resolución es de 6,25 ns a 160 MHz. El controlador evaluado es el PI del proyecto: forma posicional con integral por Tustin, $K_p = 60$, $K_i = 1$, $T_s = 10$ ms, saturación en ± 1 y anti-windup por *clamping*, en versiones `float` y `double`. Un script de Python en la PC envía cada prueba por el puerto serie, recibe los resultados en CSV y genera las figuras. Las cinco pruebas son:

- **Aritmética**: lazos de 10^5 a 10^6 iteraciones de multiplicación y suma, división y raíz cuadrada.
- **Paso del PI**: duración de 2000 pasos con entradas que imitan un escalón, descontando el costo de leer el contador.
- **Lazo periódico**: el PI se ejecuta en cada instante programado, de 1 kHz a 1 MHz, durante unos 0,5 s (hasta 20 000 periodos), y se registra el retraso de cada inicio respecto al instante ideal. Se prueba el PI en `float`, en `double` y el PI en `float` precedido de una lectura del ADC. Si un inicio se retrasa más de un periodo, se cuenta como plazo perdido y el lazo se re-sincroniza.
- **ADC**: 5000 lecturas consecutivas con `analogRead` a 12 bits.
- **Comunicación**: 2000 tramas de seis bytes, con el mismo formato del PiL, enviadas desde la PC; el ESP32-C6 calcula el PI y responde seis bytes. La PC mide el tiempo de ida y vuelta (RTT).

El benchmark completo se corrió dos veces; la aritmética y las medianas del PI coincidieron. Se reportan los datos de la segunda corrida.

III. Resultados

III-A. Aritmética

La Tabla II y la Fig. 1 muestran el costo de no tener unidad de punto flotante. Una multiplicación y suma entera tarda 25 ns, unos cuatro ciclos con la sobrecarga del lazo, mientras que en `float` tarda 533 ns, unas 21 veces más, y en `double`, 1160 ns. La división y la raíz cuadrada son todavía más costosas.

Tabla II
ARITMÉTICA A 160 MHz

Operación	ns por operación	Millones de op./s
`int32` mult. + suma	25.2	39.7
`float` mult. + suma	533.5	1.87
`double` mult. + suma	1159.6	0.86
`float` división	793.1	1.26
`double` división	2650.0	0.38
`sqrtf`	4596.4	0.22

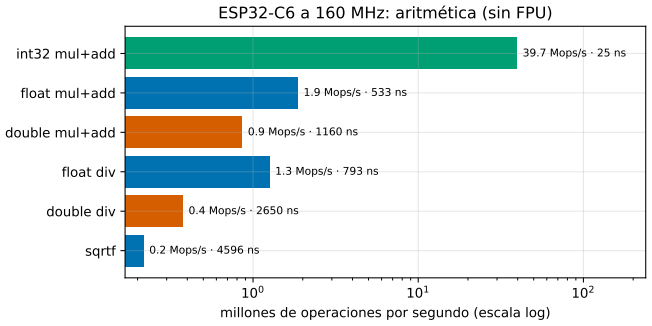


Figura 1. Rendimiento aritmético en escala logarítmica: enteros frente a punto flotante por software.

III-B. Paso del controlador PI

Un paso del PI tarda 2,66 µs en `float` (426 ciclos, mediana) y 4,51 µs en `double` (721 ciclos), como muestran la Tabla III y la Fig. 2. Los máximos, de unos 16 µs, corresponden a pocas muestras interrumpidas por el sistema operativo. Con la mediana, el tope teórico del lazo sería de 376 kHz en `float` y 222 kHz en `double`. A los 100 Hz de MARLUP, el PI en `float` ocupa el 0,027 % del periodo de 10 ms.

Tabla III
DURACIÓN DE UN PASO DEL PI (2000 PASOS)

Versión	Mediana (µs)	p99 (µs)	Máx. (µs)
`float`	2.66	2.81	16.29
`double`	4.51	12.67	15.66

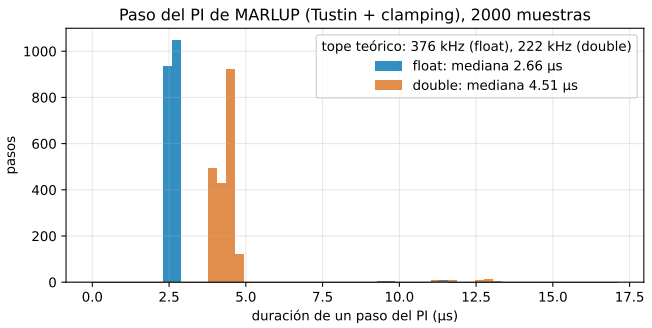


Figura 2. Distribución de la duración de un paso del PI de MARLUP.

III-C. Lazo periódico en tiempo real

La Fig. 3 y la Tabla IV resumen hasta dónde se sostiene el lazo. Con el PI en `float` no se pierde ningún plazo hasta 200 kHz, y la frecuencia lograda se satura en unos 296 kHz. Con `double`, el lazo se sostiene hasta 100 kHz y se satura en unos 198 kHz. Al agregar la lectura del ADC, el límite baja a 20 kHz sin pérdidas y a unos 25 kHz como máximo. Mientras el lazo se sostiene, el retraso típico de cada inicio es de 0,037 µs, del orden de la resolución de la medición, y el percentil 99 no pasa de 6,3 µs. Por encima del tope, el porcentaje de plazos perdidos se estabiliza cerca del 50 %: es un efecto del método, porque tras cada pérdida el lazo se resincroniza y el periodo siguiente vuelve a empezar a tiempo. Por eso la frecuencia lograda es la medida más representativa del límite.

Tabla IV
LÍMITES DEL LAZO PERIÓDICO

Variante	Sin plazos perdidos hasta	Frecuencia lograda máx.
PI `float`	200 kHz	296 kHz
PI `double`	100 kHz	198 kHz
ADC + PI `float`	20 kHz	25 kHz

III-D. ADC

Cada lectura con `analogRead` tarda 36,7 µs, es decir, unas 27 200 lecturas por segundo. Este es el cuello de botella del lazo con medición. La prueba se hizo con el pin sin conectar, por lo que solo es válida la velocidad; el ruido y la linealidad del ADC requieren una señal de referencia y no se evaluaron.

III-E. Comunicación con la PC

En las 2000 tramas, el tiempo de ida y vuelta fue de 0,156 ms en promedio (0,135 ms de mediana), con percentil 95 de 0,216 ms, percentil 99 de 0,762 ms y máximo de 3,22 ms (Fig. 4). Solo 2 tramas superaron 1 ms y ninguna se acercó al periodo de muestreo de 10 ms. El valor incluye la sobrecarga del lado de la PC (Windows y Python), así que la latencia propia del enlace USB es menor.

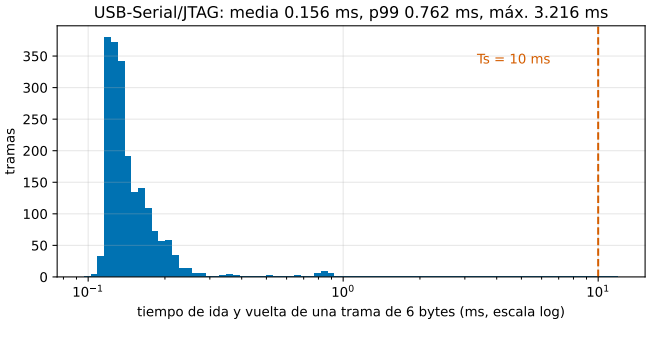


Figura 4. Tiempo de ida y vuelta de una trama de seis bytes por USB-Serial/JTAG (escala logarítmica).

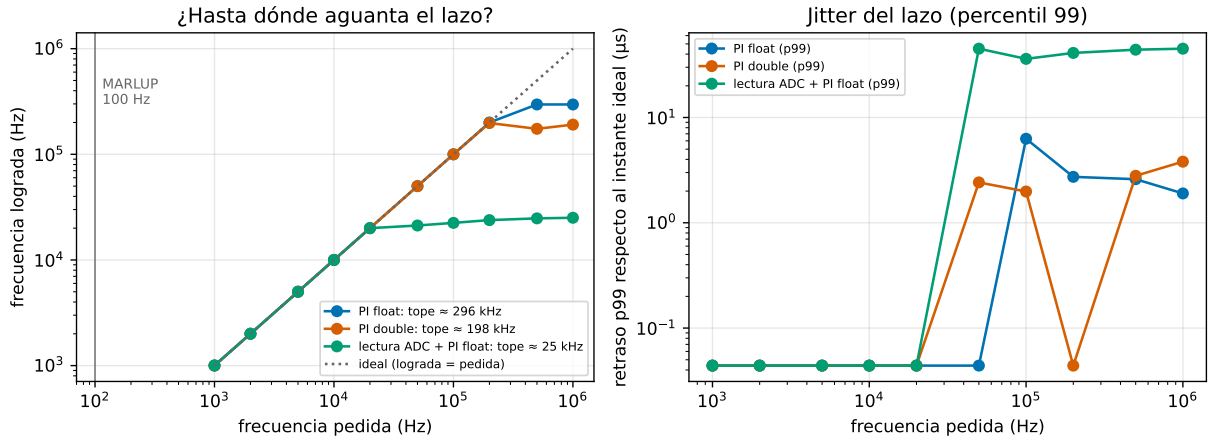


Figura 3. Lazo periódico: frecuencia lograda frente a la pedida (izquierda) y retraso del percentil 99 respecto al instante ideal (derecha). La línea vertical marca los 100 Hz de MARLUP.

IV. Discusión

Para MARLUP, que muestrea a 100 Hz, el ESP32-C6 tiene un margen muy amplio: el PI ocupa una fracción mínima del periodo y el lazo completo con lectura del ADC podría correr unas 200 veces más rápido. Tres observaciones guían su uso. Primero, conviene implementar el controlador en `float`, porque sin unidad de punto flotante `double` cuesta cerca del doble. Segundo, el límite práctico lo impone el ADC y no el cómputo; si se necesitara más velocidad, habría que usar el modo continuo del ADC en lugar de lecturas individuales. Tercero, la comunicación USB es lo bastante rápida y estable para un PiL en tiempo real. Su mayor limitación para el curso no es de desempeño: el *Simulink Support Package for Arduino Hardware* instalado (R2026a) no incluye el ESP32-C6, y sus placas ESP32 no ofrecen el modo PIL, por lo que un PiL con este chip requeriría un protocolo de comunicación propio.

V. Conclusiones

- Un paso del PI de MARLUP tarda 2,66 µs en `float` y 4,51 µs en `double`; a 100 Hz ocupa el 0,027 % del periodo.
- El lazo con el PI en `float` se sostiene sin plazos perdidos hasta 200 kHz; con lectura del ADC, hasta 20 kHz, limitado por los 36,7 µs de cada lectura.
- Sin unidad de punto flotante, las operaciones `float` cuestan unas 21 veces más que las enteras y las `double`, el doble que las `float`.
- La comunicación por USB tiene un tiempo de ida y vuelta medio de 0,156 ms y máximo de 3,22 ms, muy por debajo del periodo de 10 ms.

Como trabajo futuro se propone medir el ruido y la linealidad del ADC con una señal de referencia, correr el mismo benchmark en el Arduino UNO para una comparación directa, e implementar un PiL en tiempo real con el ESP32-C6 marcando su propio periodo de muestreo.

Apéndice

Anexo: reproducibilidad

Repositorio: https://github.com/aasalas/LAB-DE-CONTROL-GR5, carpeta `EXPERIMENTOS/esp32c6_benchmark`: firmware en `esp32c6_benchmark/`, script `correr_benchmark.py` (se ejecuta con `python correr_benchmark.py COM7`), datos en `resultados/` y figuras en `figuras/`.